

InternNova Internship

Week 4 Assignment Report

Statistics & Exploratory Data Analysis (EDA)

Name: Aryan Dubey

Task 1: Mean, Median & Mode

Brief Explanation:

This task calculates the mean, median, and mode of the Quantity column in the retail sales dataset. The mean quantity is approximately 2.548, the median is 2.0, and the mode is 1. These measures summarize the typical number of units purchased per order.

Source Code:

```
import pandas as pd

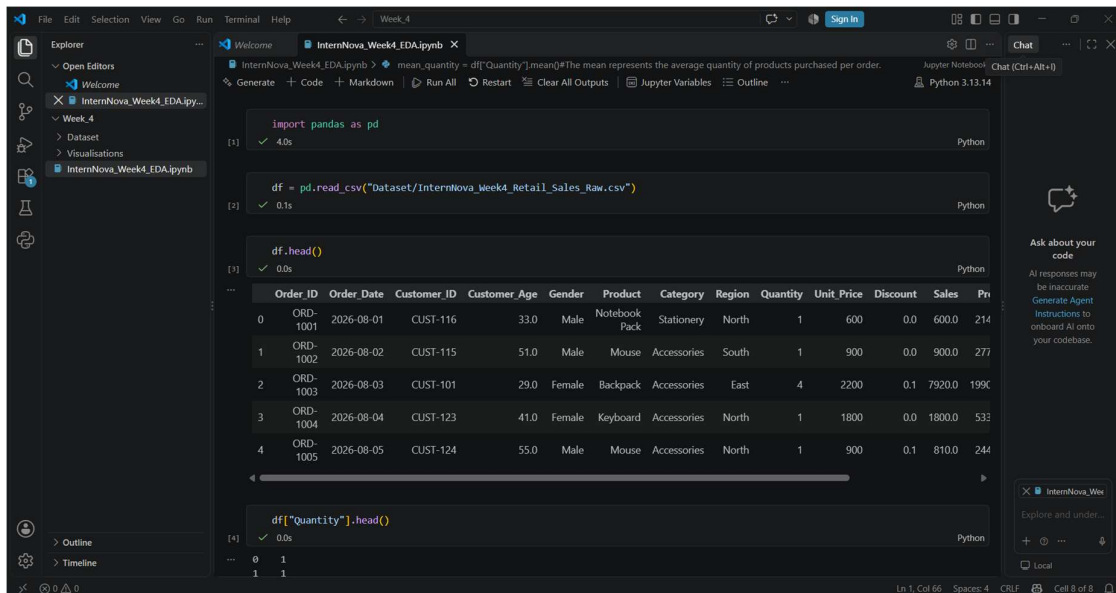
df = pd.read_csv("Dataset/InternNova_Week4_Retail_Sales_Raw.csv")

mean_quantity = df["Quantity"].mean()
median_quantity = df["Quantity"].median()
mode_quantity = df["Quantity"].mode()[0]

print("Mean Quantity:", mean_quantity)
print("Median Quantity:", median_quantity)
print("Mode Quantity:", mode_quantity)
```

OUTPUT SCREENSHOT:

Dataset Loading and Initial Preview



The screenshot displays a Jupyter Notebook environment with the following components:

- File Explorer:** Shows the project structure with folders for 'Open Editors', 'Welcome', 'InternNova_Week4_EDA.ipynb', 'Week_4', 'Dataset', 'Visualisations', and 'InternNova_Week4_EDA.ipynb'.
- Code Editor:** Contains the following Python code:

```
import pandas as pd

df = pd.read_csv("Dataset/InternNova_Week4_Retail_Sales_Raw.csv")

df.head()
```
- Output:** The output of the `df.head()` command is displayed as a table with 15 columns: Order_ID, Order Date, Customer_ID, Customer Age, Gender, Product, Category, Region, Quantity, Unit Price, Discount, Sales, and Profit. The first five rows of data are shown.
- Chat Panel:** A sidebar on the right with a 'Chat' button and a message: 'Ask about your code. AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase.'
- Status Bar:** At the bottom, it shows 'Ln 1, Col 66', 'Spaces: 4', 'CRLF', and 'Cell 8 of 8'.

Mean, Median and Mode Output

File Edit Selection View Go Run Terminal Help

Week 4

Sign In

Explorer

Open Editors

InternNova_Week4_EDA.ipynb

Week 4

Dataset

Visualisations

InternNova_Week4_EDA.ipynb

InternNova_Week4_EDA.ipynb

mean_quantity = df["Quantity"].mean()#The mean represents the average quantity of products purchased per order.

df["Quantity"].mean()

np.float64(2.5483870967741935)

df["Quantity"].median()

np.float64(2.0)

df["Quantity"].mode()

0 1

Name: Quantity, dtype: int64

mean_quantity = df["Quantity"].mean()#The mean represents the average quantity of products purchased per order.

median_quantity = df["Quantity"].median()#The median represents the middle quantity value

mode_quantity = df["Quantity"].mode()[0] # HELP IN SHOWING MOST COMMON UNIT

print("Mean Quantity:", mean_quantity)

print("Median Quantity:", median_quantity)

print("Mode Quantity:", mode_quantity)

Mean Quantity: 2.5483870967741935

Median Quantity: 2.0

Mode Quantity: 1

Chat

Ask about your code

AI responses may be inaccurate

Generate Agent

Instructions to onboard AI onto your codebase.

InternNova_Wei

Explore and under...

Local

Ln 1, Col 24 Spans: 4 CTRL (1) Cell 8 of 8

Task 2: Variance & Standard Deviation

Brief Explanation:

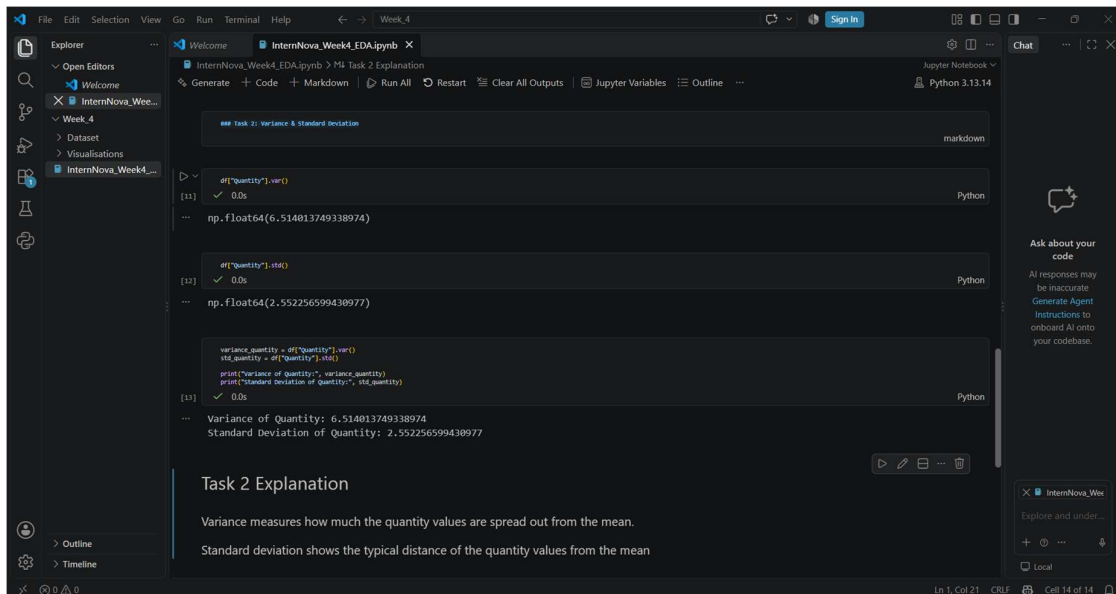
This task measures the spread of the Quantity values. The variance is approximately 6.514 and the standard deviation is approximately 2.552. Variance shows how widely quantity values are dispersed, while standard deviation shows the typical distance of the values from the mean.

Source Code:

```
variance_quantity = df["Quantity"].var()
std_quantity = df["Quantity"].std()

print("Variance of Quantity:", variance_quantity)
print("Standard Deviation of Quantity:", std_quantity)
```

OUTPUT SCREENSHOT:



Task 3: Correlation & Probability Basics

Brief Explanation:

This task calculates the correlation between Sales and Profit and the probability of selecting an order with Quantity greater than 1. Sales and Profit have a very strong positive correlation of approximately 0.997. The probability of an order containing more than one item is approximately 0.6774, or 67.74%.

Source Code:

```
correlation = df["Sales"].corr(df["Profit"])

total_orders = len(df)
multi_item_orders = (df["Quantity"] > 1).sum()
probability = multi_item_orders / total_orders

print("Correlation between Sales and Profit:", correlation)
print("Total Orders:", total_orders)
print("Orders with Quantity > 1:", multi_item_orders)
print("Probability:", probability)
print("Probability Percentage:", probability * 100, "%")
```

OUTPUT SCREENSHOT:

Correlation and Probability Calculation

The screenshot shows a Jupyter Notebook window titled "Task 3: Correlation & Probability Basics". The notebook contains three code cells. The first cell calculates the correlation between Sales and Profit, resulting in 0.997. The second cell calculates the probability of an order containing more than one item, resulting in 0.6774. The third cell prints the results. The output of the first cell is 0.997, and the output of the second cell is 0.6774. The output of the third cell is: Correlation between Sales and Profit: 0.997, Total Orders: 62, Orders with Quantity > 1: 42, Probability: 0.6774, Probability Percentage: 67.74%.

```
df["Sales"].corr(df["Profit"])
0.997

np.float64(0.9970159726287426)

The correlation between Sales and Profit is approximately 0.997, indicating a very strong positive relationship. This means that higher sales are generally associated with higher profit in the dataset.

df["Quantity"].value_counts()
Category
Accessories    20
Electronics    18
Furniture       11
Stationery       6
Appliances       6
electronics      1
Name: count, dtype: int64

total_orders = len(df)
multi_item_orders = (df["Quantity"] > 1).sum()
probability = multi_item_orders / total_orders
print("Total Orders:", total_orders)
print("Orders with Quantity > 1:", multi_item_orders)
print("Probability:", probability)
0.6774193548387096

Total Orders: 62
Orders with Quantity > 1: 42
Probability: 0.6774193548387096
```

TASK - 3 FINAL OUTPUT RESULT

Final Output and Explanation

File Edit Selection View Go Run Terminal Help

← → Week 4

Sign In

Explorer

Open Editors

Welcome

InternNova_Week4_IDA.ipynb

Week 4

Dataset

Visualizations

InternNova_Week4_...

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

Jupyter Variables

Outline

Python 3.13.14

Ask about your code

At responses may be inaccurately

Generate Agent

Instructions to onboard AI onto your codebase.

InternNova_Week4_IDA.ipynb

Explore and understand

Like

TASK - 3 FINAL OUTPUT RESULT

```
correlation = df["Sales"].corr(df["Profit"])
total_orders = len(df)
orders_with_quantity_greater_than_1 = df[df["Quantity"] > 1].sum()
probability = float(len(orders_with_quantity_greater_than_1_orders)) / total_orders

print("Correlation between Sales and Profit:", correlation)
print("Total Orders:", total_orders)
print("Orders with Quantity > 1:", len(orders_with_quantity_greater_than_1_orders))
print("Probability Percentage:", probability * 100, "%")
```

(17) ✓ 0.1s

Python

Correlation between Sales and Profit: 0.9970159726207426
Total Orders: 62
Orders with Quantity > 1: 42
Probability: 0.6774193548387096
Probability Percentage: 67.74193548387096 %

Task 3 Explanation

Correlation:
The correlation between Sales and Profit is approximately 0.997. This indicates a very strong positive correlation, meaning that higher sales are generally associated with higher profit in the dataset.

Probability:
The probability of randomly selecting an order with a quantity greater than 1 is approximately 0.6774, or 67.74%. This means that about 67.74% of the orders contain more than one item.

Ln 1, Col 21

Spaces: 4

Ctrl

Col 22 of 22

Task 4: Outlier Detection

Brief Explanation:

The IQR method is used to detect outliers in the Quantity column. Q1 is 1.0, Q3 is 3.0, and the IQR is 2.0, giving a lower bound of -2.0 and an upper bound of 6.0. Two orders are identified as potential outliers, with quantities of 12 and 18 units.

Source Code:

```
Q1 = df["Quantity"].quantile(0.25)
Q3 = df["Quantity"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

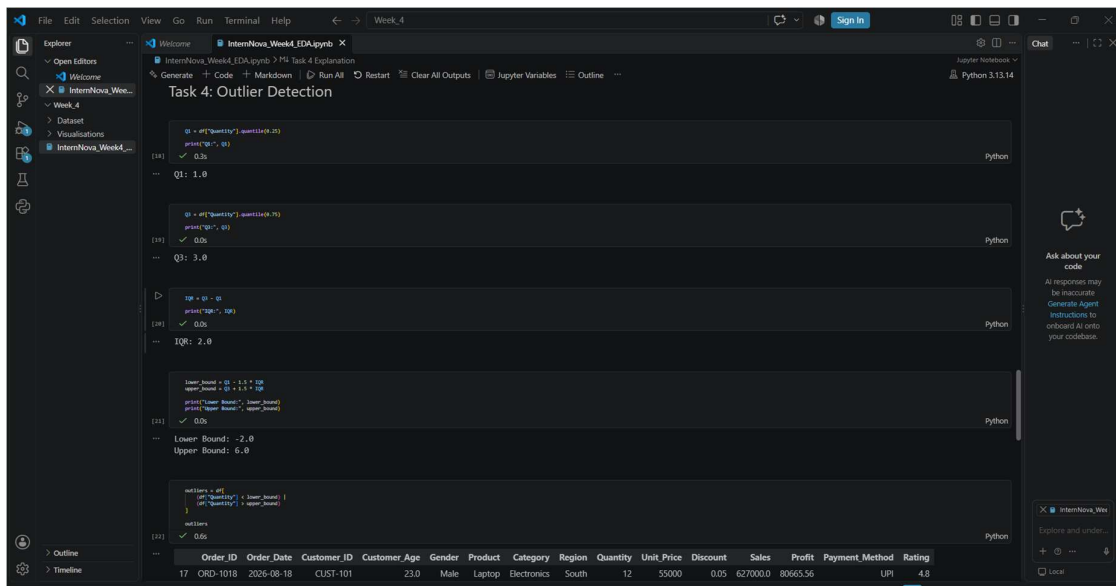
outliers = df[
    (df["Quantity"] < lower_bound) |
    (df["Quantity"] > upper_bound)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers
```

OUTPUT SCREENSHOT:

IQR Calculation and Outlier Detection



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
Q1 = df["Quantity"].quantile(0.25)
print("Q1:", Q1)
Q1: 1.0

Q3 = df["Quantity"].quantile(0.75)
print("Q3:", Q3)
Q3: 3.0

IQR = Q3 - Q1
print("IQR:", IQR)
IQR: 2.0

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
Lower Bound: -2.0
Upper Bound: 6.0

outliers = df[
    (df["Quantity"] < lower_bound) |
    (df["Quantity"] > upper_bound)
]
outliers
```

The final output shows the outliers DataFrame with two rows identified as outliers:

Order ID	Order Date	Customer ID	Customer Age	Gender	Product	Category	Region	Quantity	Unit Price	Discount	Sales	Profit	Payment Method	Rating	
17	ORD-1018	2025-08-18	CUST-101	23.0	Male	Laptop	Electronics	South	12	55000	0.05	627000.0	80665.56	UPI	4.8

Final Outlier Output and Explanation

File Edit Selection View Go Run Terminal Help

← → Week 4

Sign In

Explorer

InterNovo_Week4_IDA.ipynb

InterNovo_Week4_IDA.ipynb > P4 Task 4 Explanation

Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline

InterNovo_Week4_IDA.ipynb

Dataset

Visualizations

InterNovo_Week4_IDA.ipynb

InterNovo_Week4_IDA.ipynb

Task 4 Explanation

Python 3.13.14

ASK about your code

AI responses may be inaccurate

Generate Agent

Instructions to onboard AI into your codebase.

InterNovo_Week4_IDA.ipynb

Explore and understand

Like

TASK- 4 FINAL OUTPUT:

```
Q1 = df["Quantity"].quantile(0.25)
Q3 = df["Quantity"].quantile(0.75)
IQR = Q3 - Q1

Lower_Bound = Q1 - 1.5 * IQR
Upper_Bound = Q3 + 1.5 * IQR

outliers = df[
    (df["Quantity"] < Lower_Bound) |
    (df["Quantity"] > Upper_Bound)
]

print(f"Q1: {Q1}")
print(f"Q3: {Q3}")
print(f"IQR: {IQR}")
print(f"Lower Bound: {Lower_Bound}")
print(f"Upper Bound: {Upper_Bound}")

outliers
```

(21)

✓ 0.0s

python

Q1: 1.0

Q3: 3.0

IQR: 2.0

Lower Bound: -2.0

Upper Bound: 6.0

Order_ID	Order_Date	Customer_ID	Customer_Age	Gender	Product	Category	Region	Quantity	Unit_Price	Discount	Sales	Profit	Payment_Method	Rating	
17	ORD-1018	2026-09-18	CUST-101	23.0	Male	Laptop	Electronics	South	12	55000	0.05	627000.0	80665.56	UPI	4.8
44	ORD-1045	2026-09-14	CUST-114	51.0	Female	Monitor	Electronics	South	18	16000	0.20	230400.0	28167.22	UPI	3.7

Task 4 Explanation

The IQR method was used to detect outliers in the Quantity column.

The calculated lower bound was -2.0 and the upper bound was 6.0. Two orders were identified as potential outliers because their quantities were greater than the upper bound.

The outlier quantities were 12 and 18 units.

Outliers can affect data analysis by increasing the mean, variance, and standard deviation and may give a misleading impression of typical customer purchasing behavior. However, outliers should be investigated before being removed because they may represent genuine high-value or bulk orders.

Outline

Timeline

Ln 3, Col 17

Spans 4

280

Col 33 of 33

Task 5: Matplotlib Visualization

Brief Explanation:

This task uses Matplotlib to create five visualizations: a line chart for sales over time, a bar chart for total sales by product, a pie chart for gender distribution, a histogram for customer age, and a scatter plot for Sales versus Profit. These charts help reveal trends, category comparisons, distributions, and relationships in the retail dataset.

Source Code:

Line Chart - Sales Trend Over Time

```
import matplotlib.pyplot as plt

df["Order_Date"] = pd.to_datetime(df["Order_Date"])
df_sorted = df.sort_values("Order_Date")

plt.figure(figsize=(10, 5))
plt.plot(df_sorted["Order_Date"], df_sorted["Sales"])
plt.title("Sales Trend Over Time")
plt.xlabel("Order Date")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Bar Chart - Total Sales by Product

```
product_sales = df.groupby("Product")["Sales"].sum()

plt.figure(figsize=(10, 5))
plt.bar(product_sales.index, product_sales.values)
plt.title("Total Sales by Product")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Pie Chart - Order Distribution by Gender

```
gender_orders = df["Gender"].value_counts()

plt.figure(figsize=(6, 6))
plt.pie(gender_orders.values, labels=gender_orders.index, autopct="%1.1f%%")
plt.title("Order Distribution by Gender")
plt.show()
```

Histogram - Distribution of Customer Age

```
plt.figure(figsize=(8, 5))
plt.hist(df["Customer_Age"].dropna(), bins=10)
plt.title("Distribution of Customer Age")
plt.xlabel("Customer Age")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

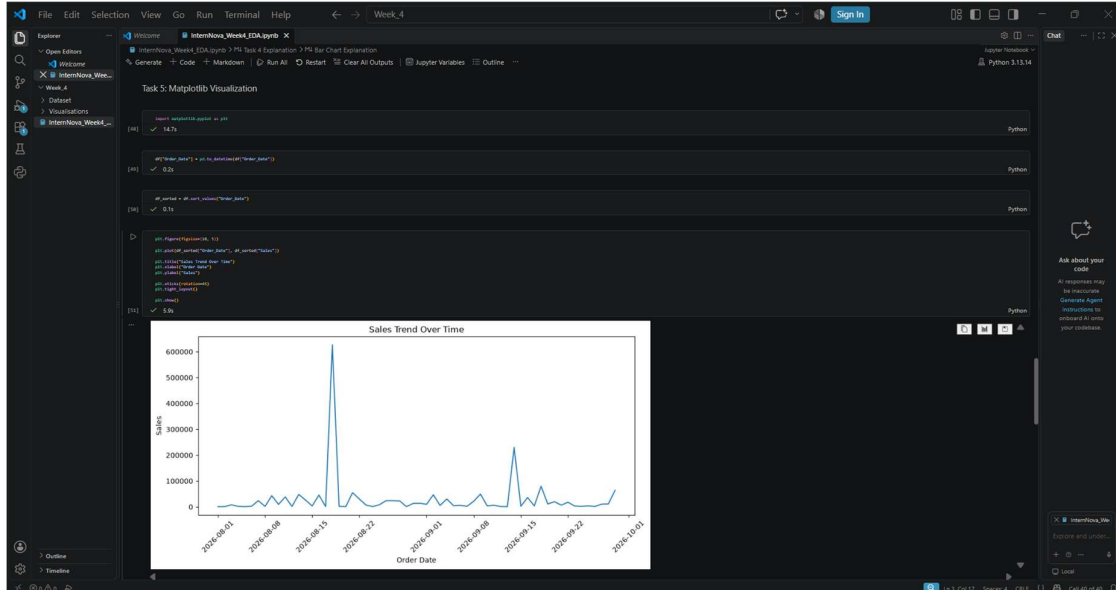
Scatter Plot - Sales vs Profit

```
plt.figure(figsize=(8, 5))
plt.scatter(df["Sales"], df["Profit"])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
```

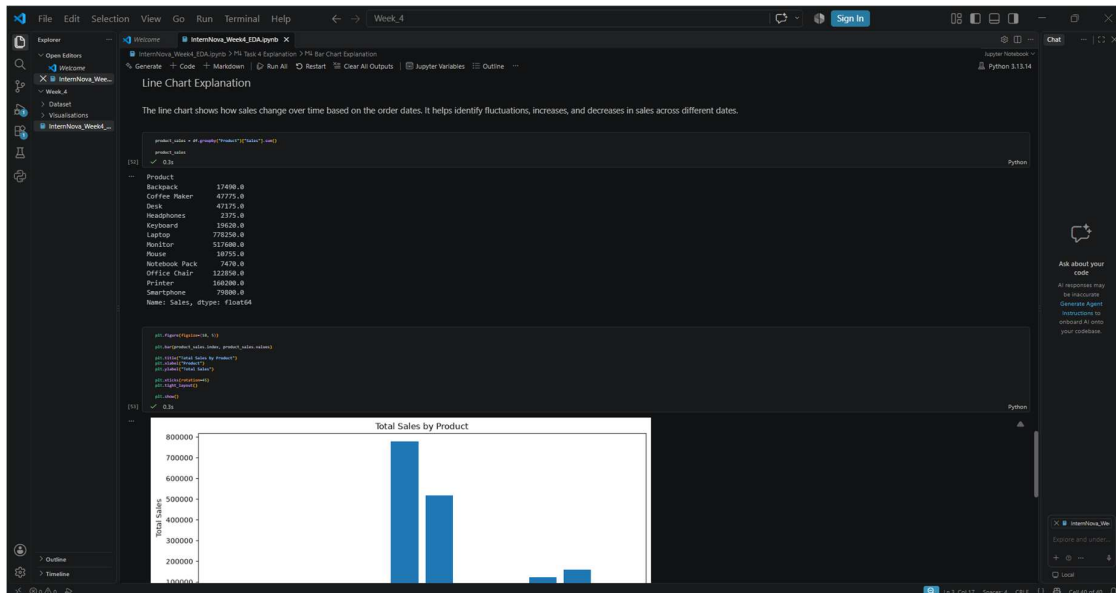
```
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```

OUTPUT SCREENSHOT:

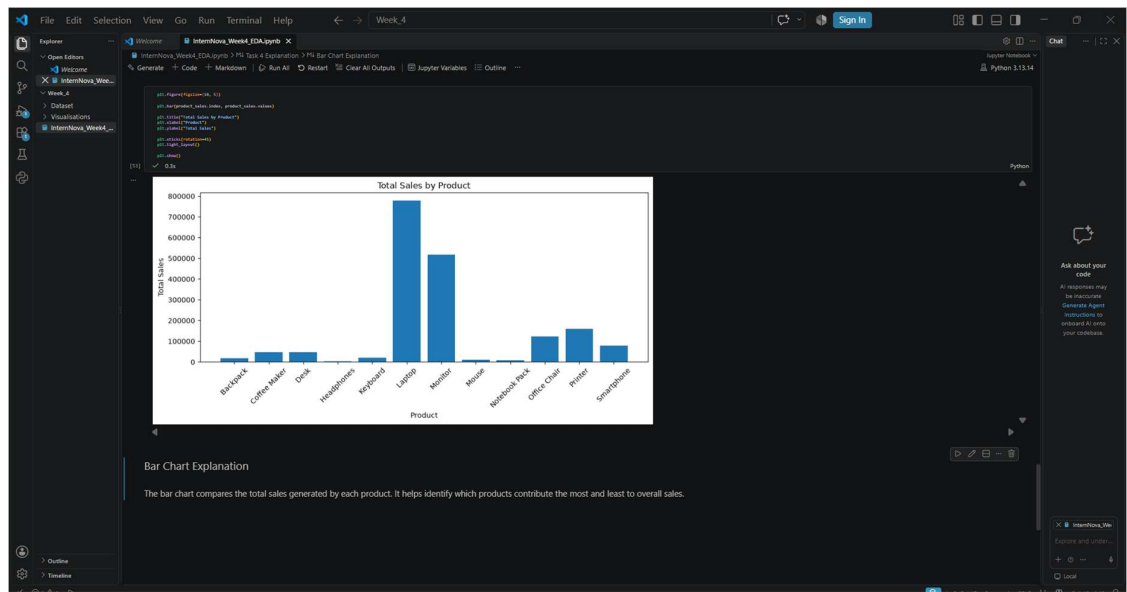
Line Chart - Sales Trend Over Time



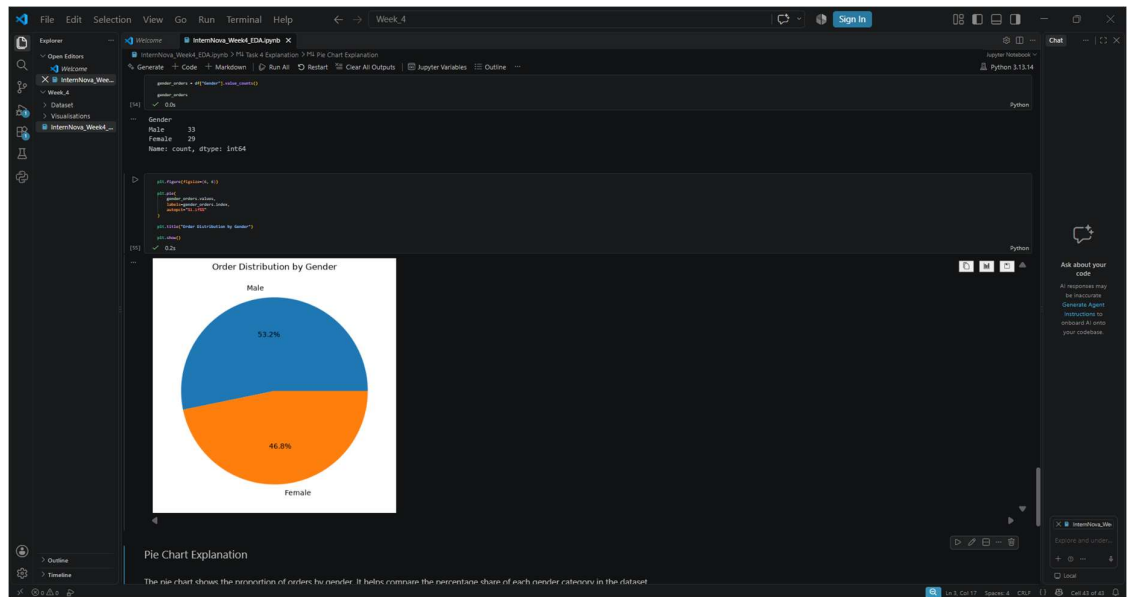
Bar Chart - Preparation and Code



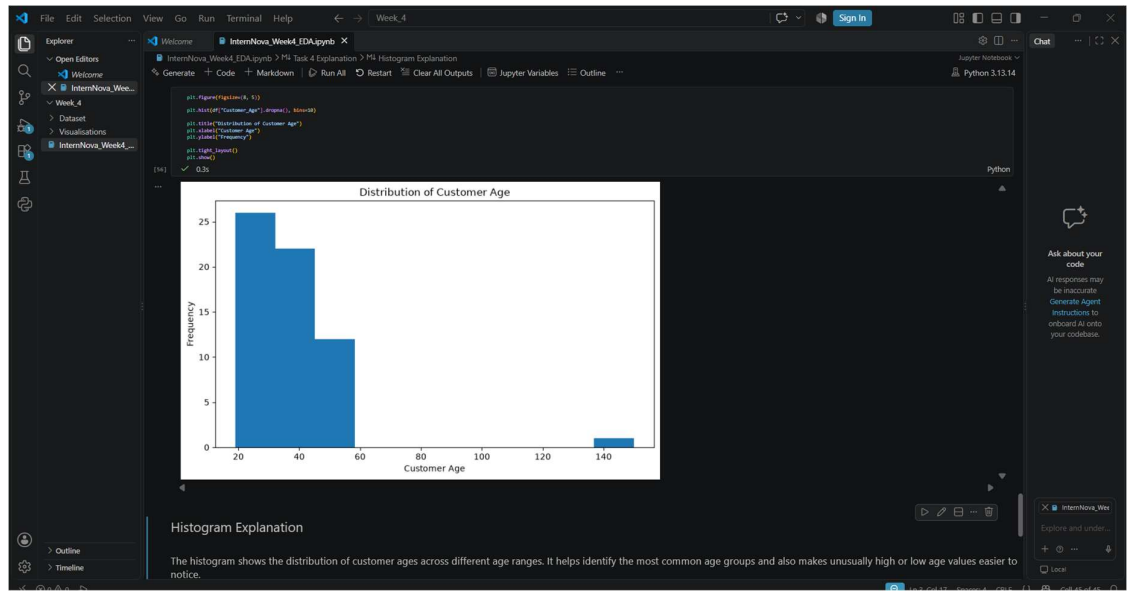
Bar Chart - Total Sales by Product



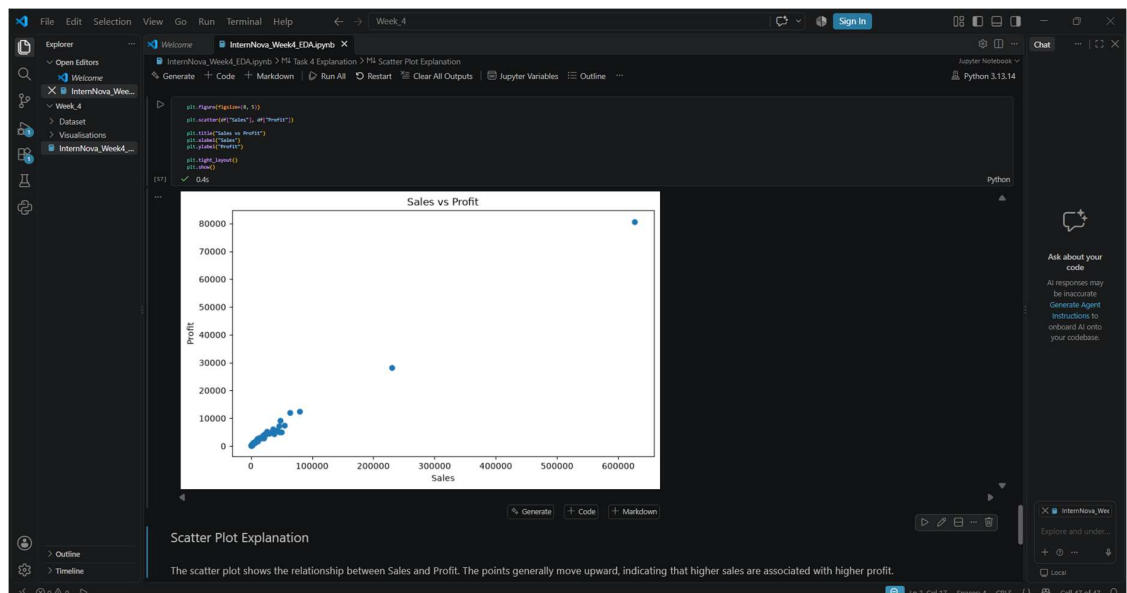
Pie Chart - Order Distribution by Gender



Histogram - Customer Age Distribution



Scatter Plot - Sales vs Profit



Task 6: Seaborn Visualization

Brief Explanation:

This task uses Seaborn for statistical visualization. A count plot compares orders by gender, a box plot highlights the distribution and outliers in Quantity, a correlation heatmap summarizes relationships among numerical variables, and a pair plot compares Quantity, Sales, Profit, and Customer Age.

Source Code:

Count Plot

```
import seaborn as sns

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Gender")
plt.title("Count of Orders by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

Box Plot

```
plt.figure(figsize=(6, 4))
sns.boxplot(data=df, y="Quantity")
plt.title("Box Plot of Quantity")
plt.ylabel("Quantity")
plt.tight_layout()
plt.show()
```

Correlation Heatmap

```
numeric_data = df.select_dtypes(include="number")
correlation_matrix = numeric_data.corr()

plt.figure(figsize=(10, 6))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

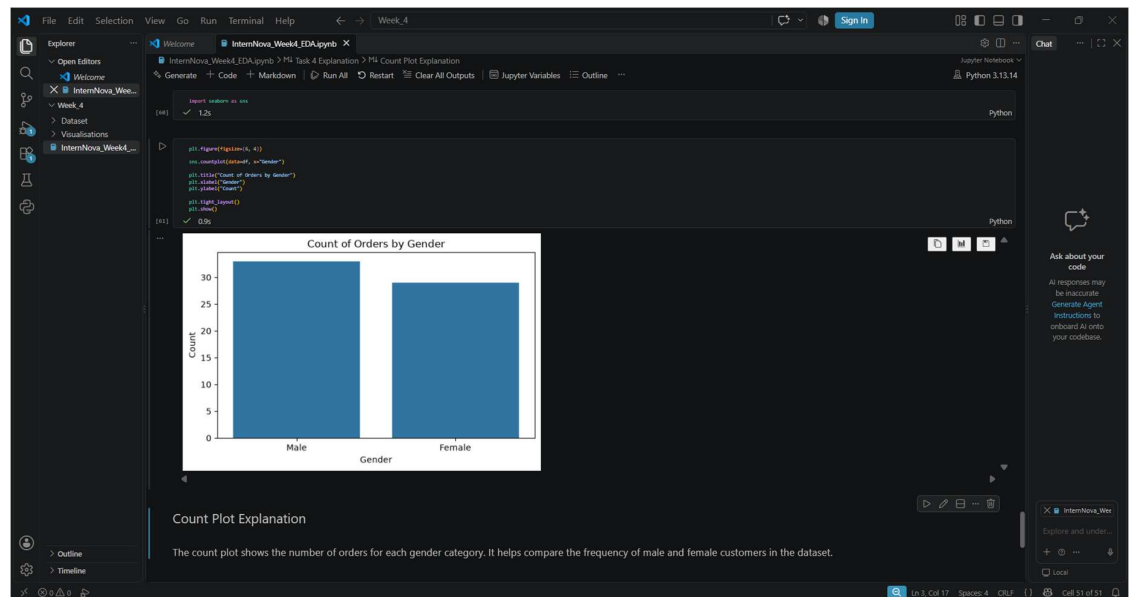
Pair Plot

```
pairplot_data = df[["Quantity", "Sales", "Profit", "Customer_Age"]].dropna()

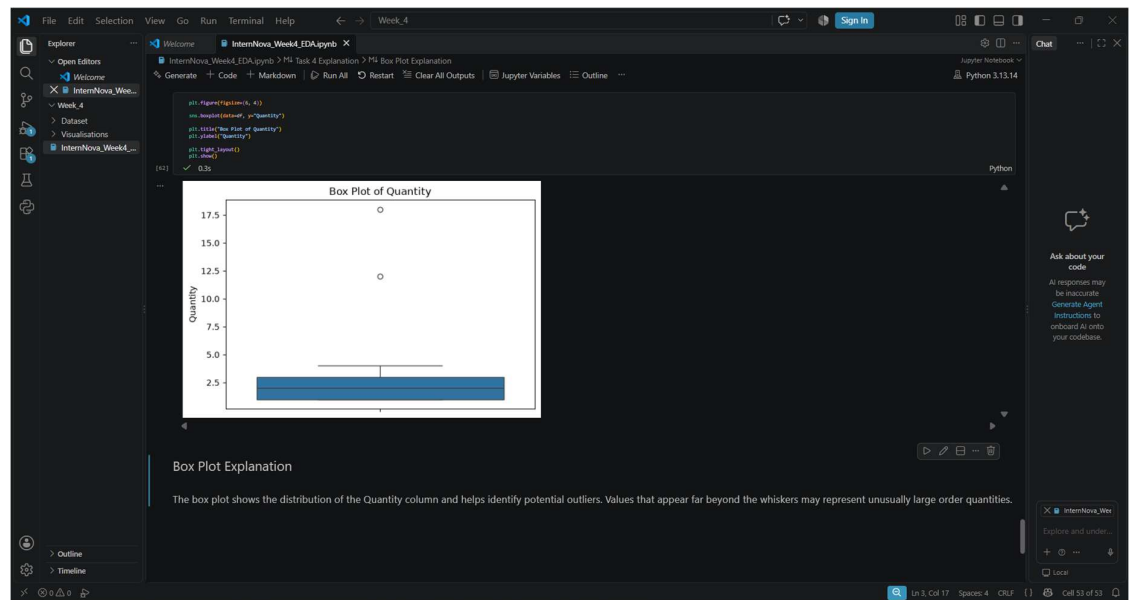
sns.pairplot(pairplot_data)
plt.show()
```

OUTPUT SCREENSHOT:

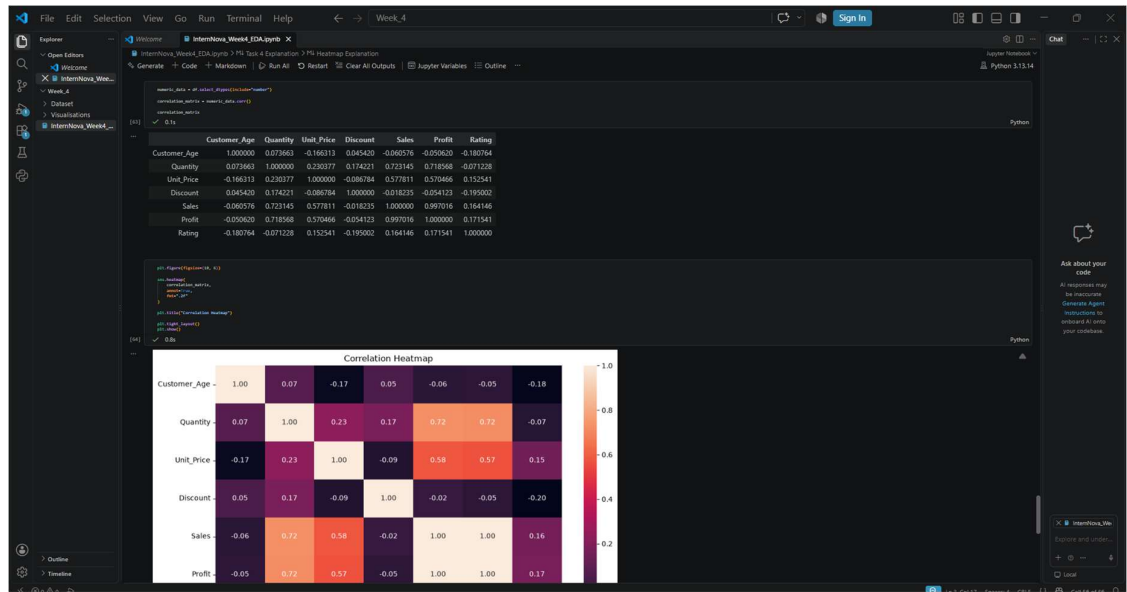
Count Plot - Orders by Gender



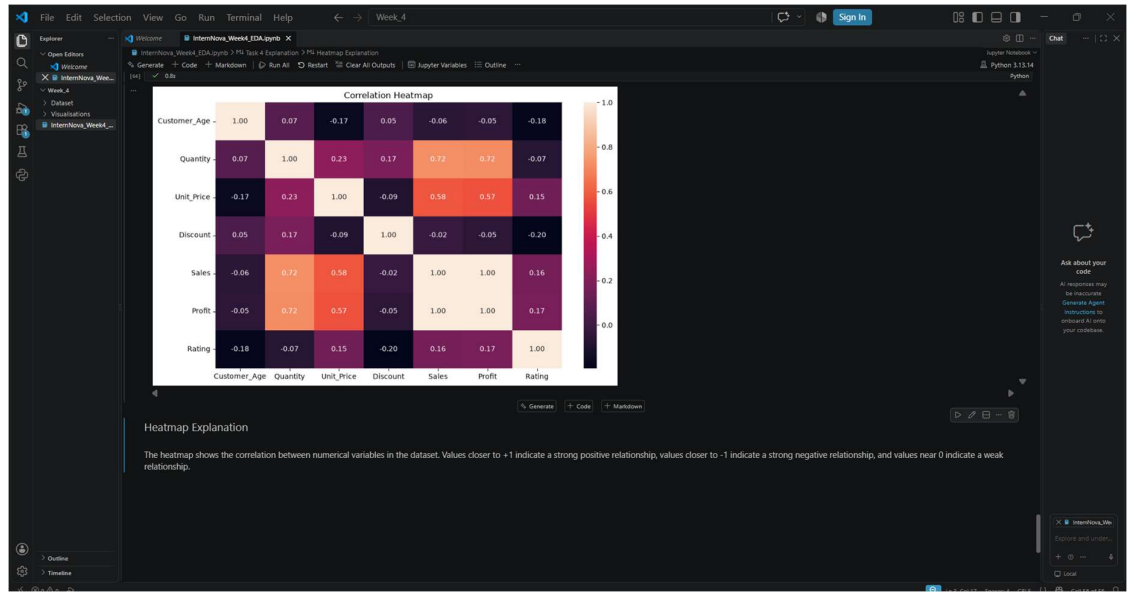
Box Plot - Quantity



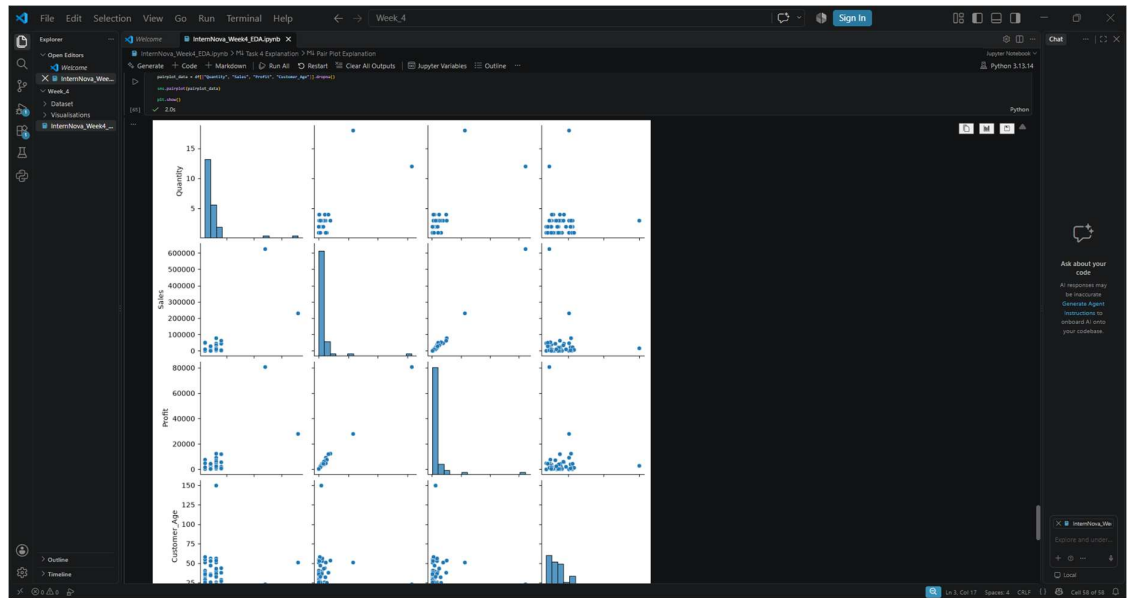
Correlation Matrix and Heatmap Code



Correlation Heatmap and Explanation



Pair Plot - Numerical Variables



Task 7: EDA - Data Inspection & Cleaning

Brief Explanation:

The dataset is inspected for shape, columns, data types, missing values, duplicate rows, unrealistic ages, and inconsistent text values. Cleaning handles missing values in Customer_Age, Region, and Rating, removes two duplicate rows, replaces the unrealistic age of 150 using the median age, and standardizes values such as electronics, north, and upi. The final cleaned dataset contains 60 rows and 15 columns with no missing values or duplicate records.

Source Code:

Inspection

```
print("Dataset Shape:", df.shape)
print(df.columns)
print(df.dtypes)
print(df.describe())

missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])

print("Duplicate Rows:", df.duplicated().sum())

print("Minimum Age:", df["Customer_Age"].min())
print("Maximum Age:", df["Customer_Age"].max())

print(df["Category"].unique())
print(df["Region"].unique())
print(df["Payment_Method"].unique())

df_before_cleaning = df.copy()

problem_rows = df[
    (df["Customer_Age"] > 100) |
    (df["Category"] == "electronics") |
    (df["Region"] == "north") |
    (df["Payment_Method"] == "upi") |
    (df.isnull().any(axis=1)) |
    (df.duplicated(keep=False))
]

problem_rows
```

Cleaning and Validation

```
df.loc[df["Customer_Age"] > 100, "Customer_Age"] = pd.NA

median_age = df["Customer_Age"].median()
df["Customer_Age"] = df["Customer_Age"].fillna(median_age)

region_mode = df["Region"].mode()[0]
df["Region"] = df["Region"].fillna(region_mode)

median_rating = df["Rating"].median()
df["Rating"] = df["Rating"].fillna(median_rating)

df = df.drop_duplicates()

df["Category"] = df["Category"].str.title()
df["Region"] = df["Region"].str.title()
```

```
df["Payment_Method"] = df["Payment_Method"].str.title()
df["Payment_Method"] = df["Payment_Method"].replace("Upi", "UPI")

print("Dataset Shape:", df.shape)
print("Total Missing Values:", df.isnull().sum().sum())
print("Duplicate Rows:", df.duplicated().sum())
print("Minimum Age:", df["Customer_Age"].min())
print("Maximum Age:", df["Customer_Age"].max())

df.to_csv("Dataset/InternNova_Week4_Retail_Sales_Cleaned.csv", index=False)
```

OUTPUT SCREENSHOT:

Before Cleaning - Problematic Rows and Inconsistent Values

The screenshot shows a Jupyter Notebook interface with a Python script that prints out the dataset's shape, missing values, duplicate rows, and minimum/maximum age. Below the script, a table of the dataset is displayed, showing columns: Order_ID, Order_Date, Customer_ID, Customer_Age, Gender, Product, Category, Region, Quantity, Unit_Price, Discount, Sales, Profit, Payment_Method, and Rating. The table contains 61 rows of data, with some rows showing missing values (NaN) for Customer_Age, Gender, Region, and Rating. The Payment_Method column shows inconsistent values like 'upi' and 'Net Banking'.

Order_ID	Order_Date	Customer_ID	Customer_Age	Gender	Product	Category	Region	Quantity	Unit_Price	Discount	Sales	Profit	Payment_Method	Rating	
6	ORD-1007	2026-08-07	CUST-111	53.0	Male	Printer	Electronics	NaN	2	12000	0.00	24000.0	4198.57	Cash	3.9
11	ORD-1012	2026-08-12	CUST-101	26.0	Female	Mouse	Accessories	East	2	900	0.20	1440.0	381.42	UPI	3.7
13	ORD-1014	2026-08-14	CUST-111	NaN	Male	Office Chair	Furniture	East	4	7000	0.05	26000.0	5202.57	Cash	3.2
20	ORD-1021	2026-08-21	CUST-122	25.0	Male	Laptop	electronics	South	1	55000	0.00	55000.0	7539.33	Credit Card	4.2
25	ORD-1026	2026-08-26	CUST-133	24.0	Male	Desk	Furniture	North	3	4500	0.05	24225.0	4626.06	Net Banking	NaN
31	ORD-1032	2026-08-01	CUST-114	40.0	Male	Coffee Maker	Appliances	north	3	3500	0.05	9975.0	2186.22	Net Banking	4.1
34	ORD-1035	2026-09-04	CUST-124	21.0	Female	Monitor	Electronics	South	2	16000	0.05	30400.0	4628.55	Credit Card	4.4
38	ORD-1039	2026-09-08	CUST-105	36.0	Female	Printer	Electronics	West	2	12000	0.10	21600.0	2923.73	upi	3.8
48	ORD-1049	2026-09-18	CUST-134	54.0	Female	Smartphone	Electronics	West	3	28000	0.05	79800.0	12372.39	Net Banking	NaN
52	ORD-1053	2026-09-22	CUST-127	150.0	Male	Office Chair	Furniture	West	3	7000	0.15	17850.0	2942.89	Cash	3.2
60	ORD-1012	2026-08-12	CUST-101	26.0	Female	Mouse	Accessories	East	2	900	0.20	1440.0	381.42	UPI	3.7
61	ORD-1035	2026-09-04	CUST-124	21.0	Female	Monitor	Electronics	South	2	16000	0.05	30400.0	4628.55	Credit Card	4.4

Cleaning Process

The screenshot shows a Jupyter Notebook interface with a Python script that performs the following cleaning steps:

- Drop rows with missing Customer_Age: `df.dropna(subset=['Customer_Age'], inplace=True)`
- Print the new shape: `print(df.shape)`
- Print the median age: `print(df['Customer_Age'].median())`
- Drop rows with missing Gender: `df.dropna(subset=['Gender'], inplace=True)`
- Print the new shape: `print(df.shape)`
- Drop rows with missing Region: `df.dropna(subset=['Region'], inplace=True)`
- Print the new shape: `print(df.shape)`
- Print the most common region: `print(df['Region'].mode()[0])`
- Drop rows with missing Rating: `df.dropna(subset=['Rating'], inplace=True)`
- Print the new shape: `print(df.shape)`
- Print the median rating: `print(df['Rating'].median())`
- Print the final shape: `print(df.shape)`

Task 8: EDA - Correlation & Insights

Brief Explanation:

This task recalculates the correlation matrix using the cleaned dataset, ranks the variables by their relationship with Sales, and visualizes the relationships with a heatmap. The strongest relationship is between Sales and Profit (approximately 0.997), followed by Quantity (approximately 0.723) and Unit Price (approximately 0.578).

Source Code:

```
numeric_data = df.select_dtypes(include="number")

correlation_matrix = numeric_data.corr()

sales_correlations = correlation_matrix["Sales"].sort_values(ascending=False)
print(sales_correlations)

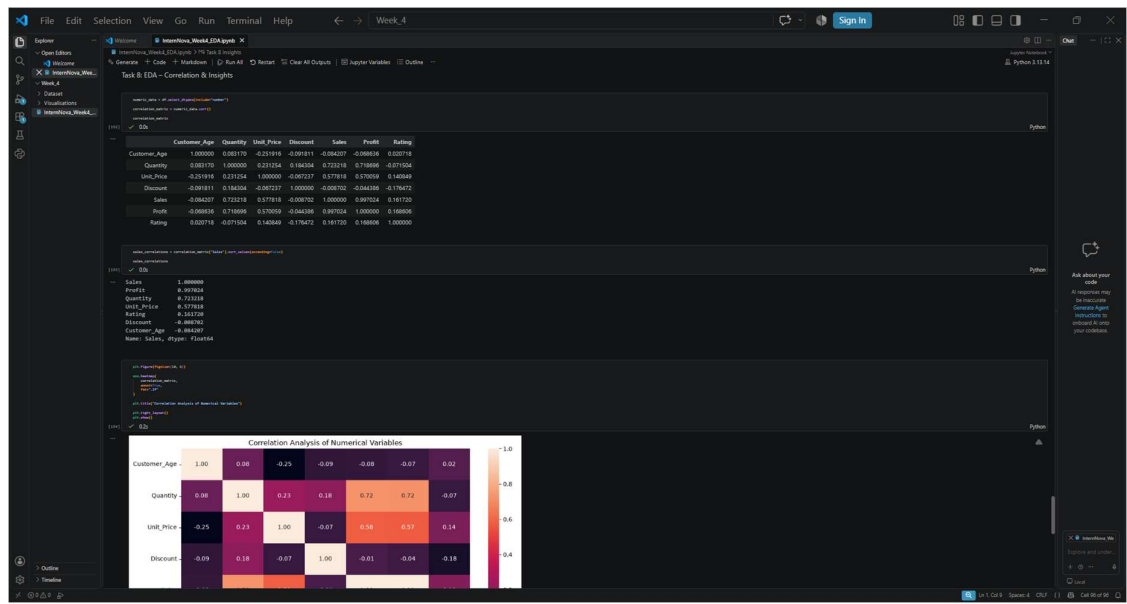
plt.figure(figsize=(10, 6))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f")
plt.title("Correlation Analysis of Numerical Variables")
plt.tight_layout()
plt.show()
```

Key Insights

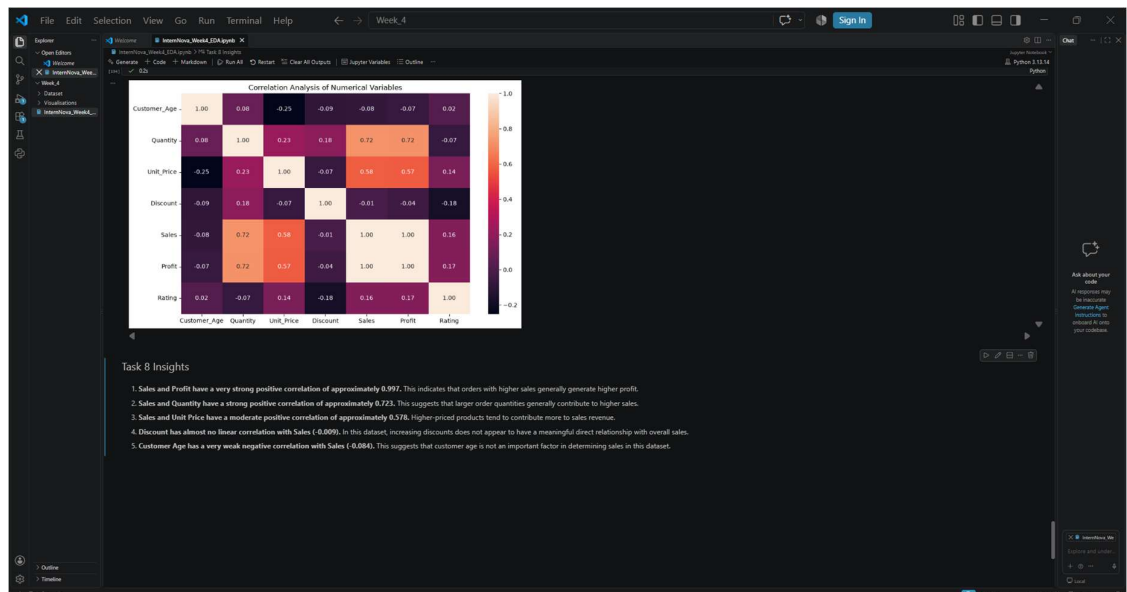
- Sales and Profit have a very strong positive correlation of approximately 0.997, indicating that orders with higher sales generally generate higher profit.
- Sales and Quantity have a strong positive correlation of approximately 0.723, suggesting that larger order quantities generally contribute to higher sales.
- Sales and Unit Price have a moderate positive correlation of approximately 0.578, so higher-priced products tend to contribute more to sales revenue.
- Discount has almost no linear correlation with Sales (-0.009), so increasing discounts does not appear to have a meaningful direct relationship with overall sales in this dataset.
- Customer Age has a very weak negative correlation with Sales (-0.084), suggesting that age is not an important factor in determining sales in this dataset.

OUTPUT SCREENSHOT:

Correlation Matrix and Heatmap



Task 8 Insights



Task 9: Business Recommendations

Brief Explanation:

This task converts the EDA findings into practical business recommendations. The recommendations are based directly on the relationships observed in the cleaned dataset, especially the strong Quantity-Sales correlation and the near-zero Discount-Sales correlation.

Business Recommendations:

Recommendation 1: Encourage Larger Order Quantities

The business should encourage customers to purchase more units through product bundles, combo offers, or quantity-based promotions.

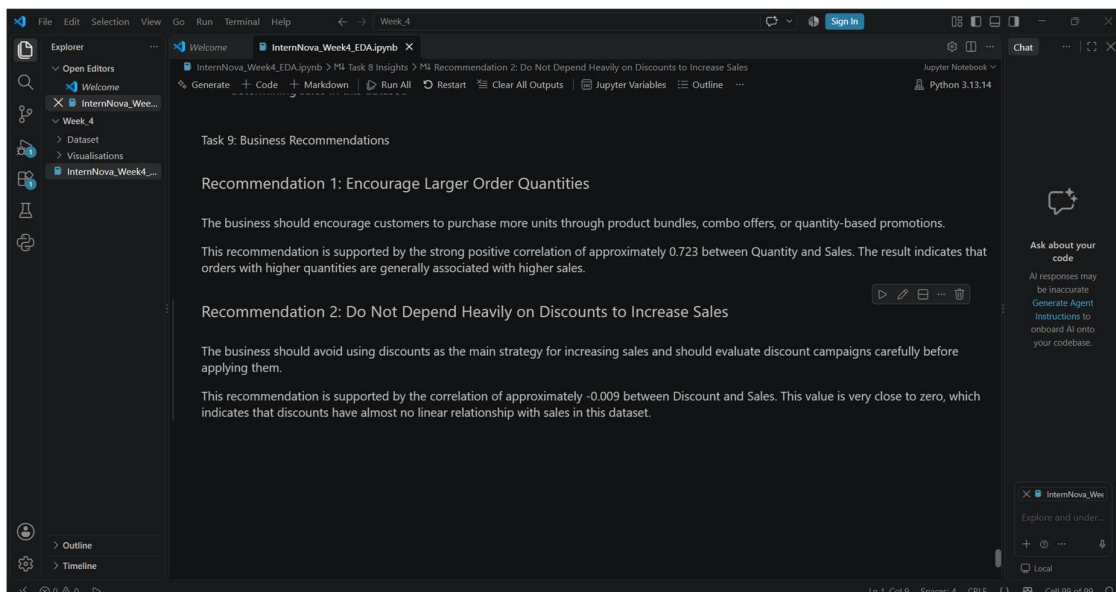
This recommendation is supported by the strong positive correlation of approximately 0.723 between Quantity and Sales. The result indicates that orders with higher quantities are generally associated with higher sales.

Recommendation 2: Do Not Depend Heavily on Discounts to Increase Sales

The business should avoid using discounts as the main strategy for increasing sales and should evaluate discount campaigns carefully before applying them.

This recommendation is supported by the correlation of approximately -0.009 between Discount and Sales. This value is very close to zero, which indicates that discounts have almost no linear relationship with sales in this dataset.

OUTPUT SCREENSHOT:



GOOGLE DRIVE LINK:

https://drive.google.com/drive/folders/1AYbDDj8_a352uXx8UMPC2UpluNcwkv6?usp=sharing